Branching Fidelity: Testing World Models as Intuitive Physics Engines for Robot Safety*

Alikasim Budhwani **Soo Jeon**
Department of Mechanical and Mechatronics Engineering, University of Waterloo, Canada
a2budhwa@uwaterloo.ca

Abstract: World models for manipulation are judged mainly by average prediction error, yet picking from a stack, pushing a tall object or inserting a peg can succeed or fail on a few millimetres of error in the start or the action. We call a state whose futures, sampled over the robot's own small errors, split into separate outcomes a *fork*, and propose the *fork probe* to evaluate a world model's *branching fidelity*: does it branch where physics branches? With a physics engine as the upper reference, the probe catches 88% of forks with 7% false alarms. Two world models with similar average error, 3–4$\times$ lower than predicting that nothing moves, differ sharply: a stochastic one catches 18% of forks, a deterministic one 60%, and 72% with uncertain starts. The probe's split test also curates training data: oversampling the rare branches it finds lifts the deterministic model from 72 to 82% of forks. Finally, in rigid-body tasks where one branch is irreversible, a fork marks the edge of the safe set, so the probe doubles as a label-free safety signal, much as people judge a stack precarious by imagining it fall.

Keywords: World models, physical fidelity, evaluation, intuitive physics

1 Introduction

World models are increasingly used to plan, evaluate and safeguard robot policies [1, 2, 3], and are judged mainly by average prediction error. Manipulation, however, is full of states where a few millimetres decide the outcome: a block picked from a stack lifts cleanly or drags a neighbour down, a pushed bottle slides or tips, a peg slides in or jams, an object nudged near a table edge stays or falls. A world model useful for such tasks must respond to small changes of state and action as physics does, and average error, dominated by states where nothing dramatic happens, does not test this.

We call a state whose futures, sampled over the robot's own small errors, split into separate outcomes a *fork*, and a world model's ability to reproduce forks its *branching fidelity*. We measure it with a *fork probe* that imagines many futures under sampled errors and tests whether they split.

Contributions. (1) The fork probe, an evaluation of branching fidelity to report beside average error, validated with a physics engine as the upper reference (Sec. 2, 3.1). (2) A benchmark showing that world models with similar average error differ sharply in branching fidelity, and why (Sec. 3.2). (3) Fork-aware training: the probe's split test curates training data and improves branching fidelity (Sec. 3.3). (4) Forks as a label-free safety signal in rigid-body manipulation, with its limits (Sec. 4).

*A version of this work also appears at the CoRL 2026 Workshop "Do Robots Need World Models?".

CoRL 2026 Workshop on Bringing Physics Simulation and World Models Together for Robotic Manipulation, Austin, TX, USA.

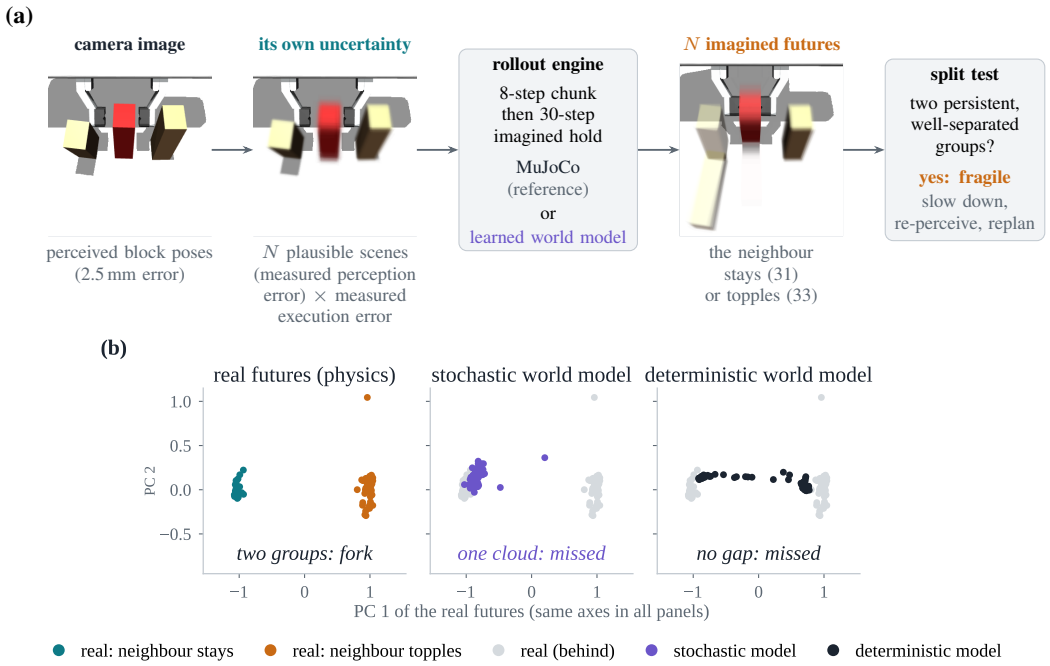


Figure 1: (a) The fork probe on one held-out state ($N = 64$; exact renders, images average all samples). (b) The N futures of that state, projected on the real futures' two main axes. The real futures form two groups (the neighbour stays or topples). The stochastic world model's futures form one cloud, so no fork is detected; the deterministic model's futures stretch along the branch but leave no gap.

2 The fork probe

Task. A simulated Franka Panda (MuJoCo) must pick the middle of three upright blocks without toppling its neighbours (Fig. 1a), a small instance of picking from a stack. A policy commits to 8-step action chunks [4, 5, 6]; the probe treats it as a black box and sees only the chunk, one camera image and the robot's joint state.

Forks. The simulated physics is deterministic, so all uncertainty is the robot's own: its error in perceiving the block poses (2.5 mm, measured on held-out images) and in executing commands (about 3 mm, measured from tracking). A state is a fork if the outcome splits over $N = 64$ futures sampled from this uncertainty: here, if between 2 and 62 of them topple a neighbour.

Probe. The probe imagines N versions of the chunk. Each starts from the perceived scene shifted by a sampled perception error and runs the chunk with a sampled execution error; the robot then holds still for 30 steps so the outcome settles. The probe flags the chunk if the N endings fall into two separate groups. We test for separate groups (bimodality), not for large spread: futures can differ smoothly without a change of outcome (a block sliding a few millimetres further), whereas a fork is a qualitative split with a gap, and a spread threshold would need tuning per task. The split test uses fixed statistical conventions, not thresholds fit to labels (Appendix A). The rollout engine can be swapped: MuJoCo given the blocks' true shapes, masses and friction, which a real robot does not know, is the *upper reference*; a learned world model is the system under test.

Ground truth. One real outcome cannot grade the probe, since a fragile action may succeed by luck. An oracle runs the N futures from the *true* scene in exact physics and labels forks as above; two independent oracle runs agree on 99% of states. Each test set has unseen block layouts, about 60 forks and 100 safe states, and was fixed before testing.

Table 1: The fork probe with a physics engine on three unseen test sets: forks caught / false alarms (%).

	Set A (153 states)		Set B (160)		Set C (156)	
	caught	false	caught	false	caught	false
pose estimate trusted (no perception sampling)	65	23	65	12	67	24
fork probe, endings read from block positions	90	12	77	8	91	10
fork probe, endings read from images	88	12	77	8	**88**	**7**[*]

[*] with 5 sampled scene estimates (Appendix A); 9% with one.

Table 2: World models. Spread: total spread of the futures relative to real futures; main axis: its share along the real futures' main axis; fork ratio: spread on forks over spread on safe states. Fresh test sets were never used for training or selection.

rollouts from	caught	false	spread	main axis	fork ratio
test set C (156 states)					
physics engine (upper reference)	88%	7%	1.00	0.57	4.8
world model, stochastic	18%	2%	0.98	0.15	1.3
world model, deterministic	60%	10%	0.04	0.69	32
two fresh test sets (160 / 152 states)					
uncertain starts + deterministic	72 / 82%	8 / 7%			
+ rare-branch oversampling	82 / 85%	9 / 11%			

3 Results

3.1 The upper reference: the probe with a physics engine

Sampling the perception error matters: a probe that trusts the pose estimate catches only 65–67% of forks, because perception errors are as large as execution errors; sampling them raises this to 77–91% (Table 1). Reading the endings from image features instead of block positions changes little: on test set C the probe catches 88% of forks (95% CI 77–94%) with 7% false alarms. This is the bar a learned world model should reach.

3.2 Benchmark: similar error, different branching fidelity

A learned model now takes MuJoCo's role. Our models predict in a compact DINOv2 feature space [7] (Appendix A); each training image comes with 8 real futures from perturbed starts, so a model can learn the robot's own uncertainty. We compare a stochastic model (flow matching), which draws fresh noise at every step, with a deterministic one that predicts the next state directly. By average error the two look alike: both are 3–4$\times$ better than predicting that nothing moves, the deterministic one slightly better (0.042 vs 0.055). The fork probe separates them: on test set C the stochastic model catches 18% of forks and the deterministic one 60% (Table 2).

Why the stochastic model fails. In physics, futures differ only because starts and commands differ, and they diverge only near the boundary between outcomes. The stochastic model instead adds noise at every step. It learns the right *amount* of spread (0.98 of the real futures') but not *where* it belongs: its futures spread in every direction, almost as much on safe states as on forks (fork ratio 1.3 vs 4.8 for physics; Fig. 1b). Five further generative variants catch at most 3% of forks, with the same shapeless spread (Appendix A).

Why the deterministic model comes closer. Its futures differ only through their perturbed inputs, so, as in physics, its spread comes from the sensitivity of its dynamics: tiny overall, but along the branch and 32$\times$ larger on forks. Its weakness is the opposite one: trained with squared error, it predicts the average of the outcomes it has seen, so near a fork its futures stretch toward both branches without a gap (Fig. 1b), and rare branches get averaged away.

3.3 Fork-aware training

The probe's ingredients also serve as a training recipe. **Uncertain starts:** uncertainty should enter where it really is, in the robot's knowledge of the scene, not in the physics. A learned operator perturbs the start by sampled perception error and the deterministic model rolls each start out. Trained on fork-rich data, this catches 72% of forks on a fresh test set and plateaus there: longer training, more data, a frame history or DINO-WM's transformer predictor [1] all land at 68–72%. A generative model given the same perturbed starts still catches 0%: the noise in its dynamics erases the forks. **Data curation with the split test:** without labels, we apply the split test to each training scene's own real futures, mark the smaller group as the rare branch, and fill half of every batch with rare-branch futures. On two fresh test sets this lifts the uncertain-start model from 72 to 82% and from 82 to 85% of forks, at 1–4 points more false alarms. Part of the gain may come from a faster training recipe this model also used. Branching fidelity can be trained for, not only measured.

4 Application: forks as a label-free safety signal

The probe mirrors how people judge physical risk. In the *intuitive physics engine* account of cognitive science [8, 9, 10], people decide whether a stack will fall by simulating it in their heads a few times, each with slightly different guesses of where things are, and call it precarious when the outcomes disagree [11]. The fork probe is this check with measured errors in place of guesses. In rigid-body manipulation one branch is often irreversible: a toppled block stays toppled, a dropped part stays dropped. A fork then marks a state one small error away from the unsafe set, i.e. the boundary of the safe set (Fig. 2), found without failure labels. Our three-block pick represents such tasks (stacking, picking from clutter). The signal is a proxy, not a failure detector, and does not suit every task: both branches of a fork may be harmless (a pushed ball rolls left or right), an action that always fails lies inside the unsafe set and never splits, and gradual harm (spills, deformation) need not split at all. A robot should pair the flag with a harm signal, e.g. whether the outcome can be undone. Our follow-up work builds a label-free safety filter for manipulation policies on this signal.

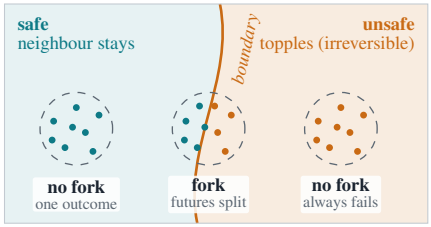


Figure 2: A fork is a state whose uncertainty straddles the boundary between safe and unsafe outcomes.

5 Related work

Robotics runs Monte Carlo rollouts over pose, friction or model parameters to score grasps and pushes [12, 13], build safety filters [14, 15] or judge whether blocks stay standing [16, 17]; Goyal [18] and Zapolsky and Drumwright [19] measure how perturbed rollouts diverge. These predict a task outcome, a stability label or a divergence rate; our probe is a per-chunk Monte Carlo test of closeness to a basin boundary [20]. Learned world models [21, 22, 1, 2, 3] struggle with stiff contact [23] and physical grounding [24, 25, 26]; Yu et al. [27] evaluate branches created by different actions, ours arise from one action under small errors. Stochastic and ensemble models [28, 29] are judged by likelihood, calibration or return. Safety filters and failure detectors use failure labels, epistemic or policy uncertainty [30, 31, 32, 33], not whether the physics is at a branch.

6 Conclusion

World models for manipulation should branch where physics branches. The fork probe measures this where average error cannot, and its split test helps train for it; benchmarks should report both. Next: pretrained world models (V-JEPA 2-AC), other tasks and a real robot.

Limitations

- **Scope.** Simulation only, one task with three blocks; other tasks, real images and real robots are untested, and there is no closed-loop intervention or latency experiment here.
- **Ground truth.** The fork label uses the same perception and execution error model that the probe samples, so it measures agreement with that model, not with real outcomes: if the robot's real errors are biased or heavier-tailed, real forks lie elsewhere. A real-robot anchor is needed. One Gaussian models perception error for all images, and the engine probe uses the simulator's true shapes, masses and friction. Sampled pose errors were applied without letting the blocks settle; a later check found that this moves a block with the robot still in 3 of 25 three-block states.
- **Split test.** It projects the endings on one main axis and fits at most two Gaussians, so it cannot represent more than two outcome regimes, can miss a split along a minor axis, and cannot see a branch reached by fewer than 2 of the N futures.
- **Operating points.** Each method is reported at one operating point fixed before testing; a sweep is possible only for the physics-engine probes on set C (Appendix B). The test sets differ in difficulty.
- **World models.** Each was trained once, at small scale, from scratch; the DINO-WM comparison re-implements its predictor on our latents rather than using the released model. The fork-rich training data of the uncertain-start models was screened with topple counts; only the rare-branch oversampling is label-free, and the oversampled model also used a faster training recipe.
- **Forks as a safety proxy.** A fork signals danger only when one branch is harmful, as for toppling rigid bodies. Forks with two harmless branches are false alarms for safety, actions that always fail are never forks, and gradual harm (deformation, spills) need not split into separate groups.

AI usage statement. Generative AI tools (Claude) assisted with code, figures, literature search and drafting. The authors designed the study, checked all results and references, and take full responsibility for the content.

References

[1] G. Zhou, H. Pan, Y. LeCun, and L. Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *International Conference on Machine Learning*, 2025.

[2] M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, M. Komeili, M. Muckley, A. Rizvi, C. Roberts, K. Sinha, A. Zholus, S. Arnaud, A. Gejji, A. Martin, F. R. Hogan, D. Dugas, P. Bojanowski, V. Khalidov, P. Labatut, F. Massa, M. Szafraniec, K. Krishnakumar, Y. Li, X. Ma, S. Chandar, F. Meier, Y. LeCun, M. Rabbat, and N. Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv:2506.09985*, 2025.

[3] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640:647–653, 2025.

[4] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023.

[5] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems*, 2023.

[6] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0: A vision-language-action flow model for general robot control. *arXiv:2410.24164*, 2024.

[7] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jegou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024.

[8] P. W. Battaglia, J. B. Hamrick, and J. B. Tenenbaum. Simulation as an engine of physical scene understanding. *Proceedings of the National Academy of Sciences*, 110(45):18327–18332, 2013.

[9] K. A. Smith and E. Vul. Sources of uncertainty in intuitive physics. *Topics in Cognitive Science*, 5(1):185–199, 2013.

[10] A. N. Sanborn, V. K. Mansinghka, and T. L. Griffiths. Reconciling intuitive physics and Newtonian mechanics for colliding objects. *Psychological Review*, 120(2):411–437, 2013.

[11] L. Zhou, K. A. Smith, J. B. Tenenbaum, and T. Gerstenberg. Mental Jenga: A counterfactual simulation model of causal judgments about physical support. *Journal of Experimental Psychology: General*, 152(8):2237–2269, 2023.

[12] J. Mahler, J. Liang, S. Niyaz, M. Laskey, R. Doan, X. Liu, J. Aparicio Ojea, and K. Goldberg. Dex-Net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics. In *Robotics: Science and Systems*, 2017.

[13] W. C. Agboh and M. R. Dogar. Pushing fast and slow: Task-adaptive planning for non-prehensile manipulation under uncertainty. In *Workshop on the Algorithmic Foundations of Robotics (WAFR)*, pages 160–176, 2018.

[14] A. Johansson, D. Lindmark, V. Wiberg, and M. Servin. Safety filtering of robotic manipulation under environment uncertainty: a computational approach. *arXiv:2509.12674*, 2025.

[15] G. Wang, K. Ren, A. S. Morgan, and K. Hang. ManiDreams: An open-source library for robust object manipulation via uncertainty-aware task-specific intuitive physics. *arXiv:2603.18336*, 2026.

[16] R. Cannizzaro, M. Groom, J. Routley, R. O. Ness, and L. Kunze. COBRA-PPM: A causal Bayesian reasoning architecture using probabilistic programming for robot manipulation under uncertainty. *arXiv:2403.14488*, 2024.

[17] A. Lerer, S. Gross, and R. Fergus. Learning physical intuition of block towers by example. In *International Conference on Machine Learning*, 2016.

[18] A. Goyal. Measuring the stability assumption behind action chunking. *arXiv:2610.01626*, 2026.

[19] S. Zapolsky and E. Drumwright. Particle traces for detecting divergent robot behavior. *arXiv:1608.01606*, 2016.

[20] P. J. Menck, J. Heitzig, N. Marwan, and J. Kurths. How basin stability complements the linear-stability paradigm. *Nature Physics*, 9:89–92, 2013.

[21] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, and P. W. Battaglia. Learning to simulate complex physics with graph networks. In *International Conference on Machine Learning*, 2020.

[22] K. R. Allen, Y. Rubanova, T. Lopez-Guevara, W. Whitney, A. Sanchez-Gonzalez, P. Battaglia, and T. Pfaff. Learning rigid dynamics with face interaction graph networks. In *International Conference on Learning Representations*, 2023.

[23] M. Parmar, M. Halm, and M. Posa. Fundamental challenges in deep learning for stiff contact dynamics. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2021.

[24] D. M. Bear, E. Wang, D. Mrowca, F. J. Binder, H.-Y. F. Tung, R. T. Pramod, C. Holdaway, S. Tao, K. Smith, F.-Y. Sun, L. Fei-Fei, N. Kanwisher, J. B. Tenenbaum, D. L. K. Yamins, and J. E. Fan. Physion: Evaluating physical prediction from vision in humans and machines. In *NeurIPS Datasets and Benchmarks*, 2021.

[25] S. Motamed, L. Culp, K. Swersky, P. Jaini, and R. Geirhos. Do generative video models understand physical principles? In *IEEE/CVF Winter Conference on Applications of Computer Vision*, 2026.

[26] T. Yang, Z. Shen, Z. Mi, Z. Zhang, J. Zhou, J. Ji, J. Dai, J. Chen, B. Chen, and Y. Yang. MiraBench: Evaluating action-conditioned reliability in robotic world models. *arXiv:2605.29360*, 2026.

[27] Y. Yu, S. Zhang, Y. Sheng, H. Ren, and H. Lin. How should world models be evaluated for embodied decision-making? A decision-making-centric position. *arXiv:2606.15032*, 2026.

[28] K. Chua, R. Calandra, R. McAllister, and S. Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, 2018.

[29] B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, 2017.

[30] K. Nakamura, L. Peters, and A. Bajcsy. Generalizing safety beyond collision-avoidance via latent-space reachability analysis. In *Robotics: Science and Systems*, 2025.

[31] J. Seo, K. Nakamura, and A. Bajcsy. Uncertainty-aware latent safety filters for avoiding out-of-distribution failures. In *Conference on Robot Learning*, 2025.

[32] C. Xu, T. K. Nguyen, E. Dixon, C. Rodriguez, P. Miller, R. Lee, P. Shah, R. Ambrus, H. Nishimura, and M. Itkina. Can we detect failures without failure data? Uncertainty-aware runtime failure detection for imitation learning policies. In *Robotics: Science and Systems*, 2025.

[33] C. Agia, R. Sinha, J. Yang, Z.-a. Cao, R. Antonova, M. Pavone, and J. Bohg. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. In *Conference on Robot Learning*, 2024.

[34] K. A. Ashman, C. M. Bird, and S. E. Zepf. Detecting bimodality in astronomical datasets. *The Astronomical Journal*, 108:2348–2361, 1994.

A Implementation details

Uncertainty model. Execution error is drawn from 64 snippets of the robot's measured tracking residuals (median per-chunk maximum 3.2 mm). Perception is a frozen DINOv2 encoder on one close-camera image with a small head that predicts the three blocks' poses (2.5 mm held-out error); its error model is a zero-mean Gaussian fitted once, offline, to its errors on held-out images with known poses (a full 9×9 position covariance capturing correlated errors, plus rotation covariances).

Actions and the hold. An action is an absolute end-effector position target plus a gripper command, tracked by an impedance controller for 0.1 s. Chunk step k executes the commanded target plus the sampled execution error; during the 3 s hold the arm holds the chunk's last nominal target, without execution error. The world models receive exactly the same action sequence, with no special hold input.

Split test. The 64 endings (block positions, or DINOv2 features of the rendered endings) are projected on their first principal axis at hold steps 10 and 30. The chunk is flagged if, at both times, two Gaussians fit better than one (BIC), the groups are well separated (Ashman's $D > 2$ [34]) and the smaller group has at least 2 members, and the grouping is the same at both times. These are fixed statistical conventions, not fitted to labels.

Five scene hypotheses. Each hypothesis draws one scene estimate from the perception-error model around the perceived scene; its 64 futures add a fresh perception draw and an execution-error sample each. The probe flags if at least 3 of the 5 hypotheses split.

Ground truth. The oracle is the probe centred on the true scene: each of its 64 futures starts from the true block poses shifted by a fresh perception-error draw and uses one execution-error sample (the same 64 snippets as the probe, different perception seeds). A neighbour topples if it tilts more than 45° by the end. Test-set states were found by an execution-only screen, not by the oracle.

World models. The latent compresses DINOv2 patch tokens by a fixed per-patch PCA to $28 \times 28 \times 8$; applied to real futures, the split test on this latent still catches 97% of forks. The dynamics are a convolutional network with six residual blocks conditioned on the action. One model step covers two simulator actions (0.2 s), so the 8-step chunk plus the 30-step hold is 19 model steps; the quoted prediction error is the open-loop latent error at the end of this horizon (0.055 stochastic, 0.042 deterministic, 0.176 for predicting that nothing moves, i.e. copying the current latent). The stochastic model is a flow-matching transition that draws fresh noise at every step; the deterministic one predicts the next latent directly. Each training image comes with 8 real futures whose starts were perturbed by sampled perception error.

Generative variants. Whole-clip generation, an energy-score loss, 4× more fork data, and an action-sensitivity loss. On 160 held-out states from the training domain (75 forks) they catch 0–3% of forks at 0–1% false alarms, with spread 1.1–1.6× that of the real futures and only 7–10% of it on the main axis.

Uncertain starts. A learned operator perturbs the start latent by a sampled perception error and the deterministic network rolls each perturbed start out. On the 160 training-domain states it catches 52% of forks at 7% false alarms, and 69% at 8% with 4× more fork-rich data; there, the perturbations were paired with the real futures' own draws. On the fresh test sets every draw is independent, as in deployment. Variants with 3× longer training, more fork data, a 3-frame history or DINO-WM's transformer predictor [1] reach 68–72% on the training-domain states.

Rare-branch oversampling. For every training state, the split test is applied to the latents of its real futures; if it splits, the smaller group at hold step 30 is the rare branch. Each batch draws half its futures from rare branches and half from the other branch of the same starts. Model and loss are unchanged, but this model was trained with a faster recipe (larger batches, the same number of samples); a model without oversampling was not retrained with that recipe. The fresh test sets have 160 states (67 forks) and 152 states (55 forks), with reset seeds disjoint from training but chunks from the same 100 demonstrations; neither was used for training or model selection. The first had earlier served as a held-out check for other models; the second was generated for this check.

B Operating-point sweep

Recomputed from the stored set-C rollouts, the defaults reproduce Table 1 exactly. Across every Ashman's-D threshold from 0.5 to 8, the fork probe catches 88–95% of forks at 8–10% false alarms, while the probe that ignores perception uncertainty never catches more than 72%, and only 58% once its false alarms fall to 8%. Results barely move between $D = 1.5$ and 4, so the default is not a tuned optimum. The minimum group size is the most sensitive setting: requiring at least 8 members per group halves every method's catch rate. With five scene hypotheses, requiring 2, 3 or 4 of the 5 to

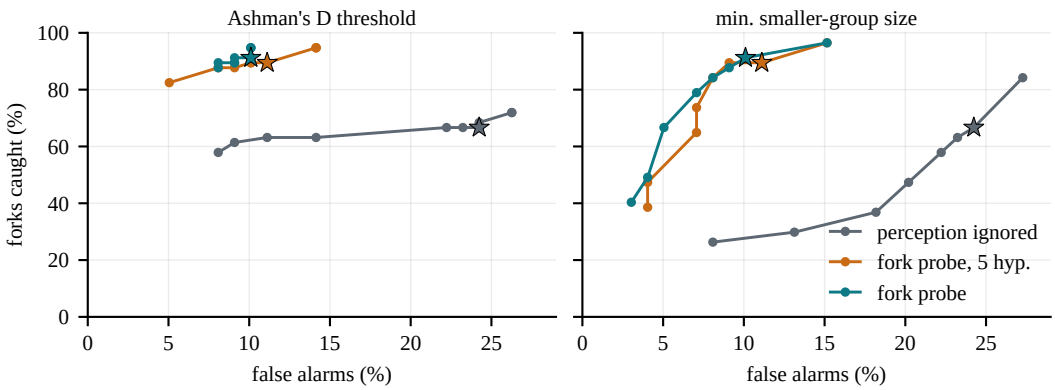


Figure 3: Forks caught vs false alarms on test set C (57 forks, 99 safe states) as one split-test setting is varied with the others at their defaults; stars mark the defaults used in Table 1. Left: Ashman's D threshold (0.5–8). Right: minimum size of the smaller group (1–16).

split gives 95/21, 89/11 and 84/7% (caught / false); adding a margin to the BIC comparison changes little up to +10. The probes' advantage holds at every operating point. Rollouts for sets A and B and for the world models were not stored, so this sweep covers the physics-engine probes on set C only.